МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение высшего образования

«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

(Университет ИТМО)

ОТЧЁТ

по лабораторной работе №1

«Кодирование и шифрование»

по дисциплине «Практическая линейная алгебра»

Выполнили:

студент группы R3243 Осипов А. П.

студент группы R3238 Федорец Л. Б.

Проверил:

Догадин Е. В.

Санкт-Петербург
2026

Содержание

Введение **3**

1 Задание 1. Шифр Хилла **3**

1.1 Алфавит и исходное сообщение 3

1.2 Метод шифрования Хилла 4

1.3 Матрицы-ключи и их определители 4

1.4 Шифрование сообщения 5

1.5 Обратные матрицы по модулю 5

1.6 Вредоносное вмешательство и дешифрация 7

Вывод по заданию 1 8

2 Задание 2. Взлом шифра Хилла **9**

2.1 Постановка 9

2.2 Исходные данные 9

2.3 Вывод способа нахождения ключа 9

2.4 Нахождение ключа 10

2.5 Проверка и расшифровка первого сообщения 11

Вывод по заданию 2 11

3 Задание 3. Код Хэмминга **12**

3.1 Алфавит и исходное сообщение 12

3.2 Построение матриц G и H 13

3.3 Кодирование сообщения 15

3.4 Вредоносное вмешательство 16

3.5 Декодирование по синдрому 18

3.6 Разбор результатов серии A 20

Вывод по заданию 3 22

Заключение **24**

Приложение A. Код **26**

Введение

В работе рассматривается применение матричного умножения к двум задачам: шифрованию сообщений (шифр Хилла) и помехоустойчивому кодированию (код Хэмминга). Основная особенность работы состоит в том, что все вычисления выполняются не над полем вещественных чисел, а над кольцом вычетов $\mathbb{Z}_n$ (задания 1 и 2) и над полем $\mathbb{F}_2$ (задание 3). Это меняет смысл привычных операций: деление заменяется умножением на обратный элемент, а обратимость матрицы определяется не условием $\det \neq 0$, а условием взаимной простоты определителя с модулем.

Вычисления выполнены на языке Python с использованием библиотеки `sympy` (целочисленная матричная арифметика, вычисление определителя и присоединённой матрицы). Обратный элемент по модулю вычисляется собственной реализацией расширенного алгоритма Евклида. Полный текст программы приведён в приложении А.

1 Задание 1. Шифр Хилла

1.1 Алфавит и исходное сообщение

В качестве алфавита взяты 26 букв латиницы и три служебных символа (_, !, ?), всего $n = 29$ символов, пронумерованных от 0 до 28. Число $n = 29$ простое, и это выбрано намеренно: при простом модуле любой определитель, не кратный n, автоматически взаимно прост с n, поэтому проверка обратимости ключа сводится к условию $\det K \not\equiv 0 \pmod{29}$. Соответствие символов и номеров приведено в таблице 1.

Таблица 1 – Соответствие символов алфавита и их числовых номеров

Символ	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
Номер	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Символ	P	Q	R	S	T	U	V	W	X	Y	Z	_	!	?	
Номер	15	16	17	18	19	20	21	22	23	24	25	26	27	28	

Шифруемое сообщение — `I_LOVE_ITMO!` длиной 12 символов. Длина выбрана не случайно: 12 делится на 2, 3 и 4, то есть на размеры всех трёх ключей, и последний блок ни при одном из них не приходится дополнять

служебными символами. Числовое представление сообщения приведено в таблице 2.

Таблица 2 – Исходное сообщение и его числовое представление

Символ	I	_	L	O	V	E	_	I	T	M	O	!
Номер	8	26	11	14	21	4	26	8	19	12	14	27

1.2 Метод шифрования Хилла

Сообщение переводится в последовательность номеров символов, которая разбивается на блоки длины z, где z — размерность квадратной матрицы-ключа K. Каждый блок рассматривается как вектор-столбец $v \in \mathbb{Z}_n^z$ и шифруется умножением на ключ по модулю n:

$$c = Kv \bmod n. \tag{1}$$

Дешифрация выполняется умножением на обратную по модулю матрицу:

$$v = K^{-1}c \bmod n. \tag{2}$$

Приведение по модулю необходимо, чтобы результат оставался в диапазоне $[0, n-1]$, то есть соответствовал какому-либо символу алфавита.

Существенно, что блоки обрабатываются независимо друг от друга: матрица умножается на каждый вектор отдельно, и никакого переноса информации между блоками не происходит. Например, фрагмент `I_LOVE` с номерами $(8, 26, 11, 14, 21, 4)$ при ключе размера 2 разбивается на блоки $(8, 26)$, $(11, 14)$, $(21, 4)$, а при ключе размера 3 — на блоки $(8, 26, 11)$ и $(14, 21, 4)$. Это свойство определяет и поведение шифра при искажениях, что разбирается в подразделе 1.6.

1.3 Матрицы-ключи и их определители

Выбраны три матрицы-ключа размеров 2×2, 3×3 и 4×4:

$$K_2 = \begin{pmatrix} 1 & 1 \\ 2 & 8 \end{pmatrix}, \qquad K_3 = \begin{pmatrix} 7 & 0 & 28 \\ 4 & 4 & 4 \\ 8 & 1 & 2 \end{pmatrix}, \tag{3}$$

$$K_4 = \begin{pmatrix} 3 & 3 & 17 & 18 \\ 22 & 9 & 1 & 15 \\ 10 & 7 & 8 & 8 \\ 20 & 11 & 7 & 20 \end{pmatrix}. \tag{4}$$

Определитель матрицы K_2 вычисляется непосредственно:

$$\det K_2 = 1 \cdot 8 - 1 \cdot 2 = 6. \tag{5}$$

Определители K_3 и K_4 вычислены по аналогии (разложением по строке) и равны $\det K_3 = -756$ и $\det K_4 = 5285$. Поскольку работа ведётся в $\mathbb{Z}_{29}$, значение имеют не сами определители, а их вычеты по модулю 29:

$$\det K_2 \equiv 6, \qquad \det K_3 \equiv 27, \qquad \det K_4 \equiv 7 \pmod{29}. \tag{6}$$

Все три вычета отличны от нуля и попарно различны, как того требует задание. Так как 29 — простое число, из $\det K \not\equiv 0$ сразу следует $\gcd(\det K, 29) = 1$, то есть все три ключа обратимы по модулю 29.

1.4 Шифрование сообщения

Покажем расчёт на первом блоке для ключа K_2. Блок `I_` имеет числовое представление $(8, 26)$, и по формуле (1):

$$K_2 \begin{pmatrix} 8 \\ 26 \end{pmatrix} = \begin{pmatrix} 1 \cdot 8 + 1 \cdot 26 \\ 2 \cdot 8 + 8 \cdot 26 \end{pmatrix} = \begin{pmatrix} 34 \\ 224 \end{pmatrix} \equiv \begin{pmatrix} 5 \\ 21 \end{pmatrix} \pmod{29}, \tag{7}$$

поскольку $34 = 1 \cdot 29 + 5$ и $224 = 7 \cdot 29 + 21$. Номерам 5 и 21 соответствуют символы `F` и `V`, что даёт первые два символа шифрограммы. Остальные блоки и остальные ключи обработаны по той же схеме программно; полученные шифрограммы приведены в таблице 3.

1.5 Обратные матрицы по модулю

Обратная матрица по модулю n вычисляется по формуле

$$K^{-1} \equiv (\det K)^{-1} \cdot \operatorname{adj} K \pmod{n}, \tag{8}$$

где $\operatorname{adj} K$ — присоединённая матрица (транспонированная матрица алгебраических дополнений), а $(\det K)^{-1}$ — обратный элемент к определителю в $\mathbb{Z}_n$. Функция `numpy.linalg.inv` здесь неприменима: она возвращает вещественную обратную матрицу, не имеющую отношения к арифметике вычетов.

Обратный элемент находится расширенным алгоритмом Евклида. Он даёт такие целые x и y, что $ax + ny = \gcd(a, n)$; при $\gcd(a, n) = 1$ приведение этого равенства по модулю n обнуляет слагаемое ny и оставляет $ax \equiv 1$, то есть искомым обратным элементом является коэффициент x. Для $\det K_2 = 6$:

$$29 = 4 \cdot 6 + 5, \tag{9}$$

$$6 = 1 \cdot 5 + 1, \tag{10}$$

откуда обратным ходом

$$1 = 6 - 1 \cdot 5 = 6 - (29 - 4 \cdot 6) = 5 \cdot 6 - 1 \cdot 29, \tag{11}$$

то есть $6^{-1} \equiv 5 \pmod{29}$. Аналогично получено $(\det K_3)^{-1} \equiv 14$ и $(\det K_4)^{-1} \equiv 25$.

Присоединённая матрица для K_2 равна $\operatorname{adj} K_2 = \begin{pmatrix} 8 & -1 \\ -2 & 1 \end{pmatrix}$, и по формуле (8)

$$K_2^{-1} \equiv 5 \cdot \begin{pmatrix} 8 & -1 \\ -2 & 1 \end{pmatrix} = \begin{pmatrix} 40 & -5 \\ -10 & 5 \end{pmatrix} \equiv \begin{pmatrix} 11 & 24 \\ 19 & 5 \end{pmatrix} \pmod{29}. \tag{12}$$

Для остальных ключей вычисления выполнены по той же формуле программно:

$$K_3^{-1} \equiv \begin{pmatrix} 27 & 15 & 27 \\ 17 & 18 & 16 \\ 14 & 18 & 15 \end{pmatrix}, \qquad K_4^{-1} \equiv \begin{pmatrix} 22 & 8 & 7 & 12 \\ 25 & 6 & 7 & 5 \\ 27 & 14 & 23 & 14 \\ 7 & 7 & 13 & 21 \end{pmatrix} \pmod{29}. \tag{13}$$

Корректность обратных матриц проверена непосредственно:

произведение ключа на найденную обратную матрицу по модулю 29 должно давать единичную матрицу. Для K_2:

$$K_2 K_2^{-1} = \begin{pmatrix} 1 & 1 \\ 2 & 8 \end{pmatrix} \begin{pmatrix} 11 & 24 \\ 19 & 5 \end{pmatrix} = \begin{pmatrix} 30 & 29 \\ 174 & 88 \end{pmatrix} \equiv \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \pmod{29}. \tag{14}$$

Для ключей K_3 и K_4 проверка выполнена программно и также даёт единичную матрицу соответствующего размера (см. листинг 2). Дешифрация шифрограмм без внесения искажений во всех трёх случаях восстанавливает исходное сообщение `I_LOVE_ITMO!` посимвольно.

1.6 Вредоносное вмешательство и дешифрация

В каждой из трёх шифрограмм заменены символы на позициях 1, 2 и 4 (нумерация с единицы) на символы `Z`, `X` и `C` соответственно. Позиции выбраны одинаковыми для всех трёх ключей намеренно: при таком выборе единственным различием между тремя экспериментами остаётся размер блока, что делает результаты сравнимыми между собой. В таблице 3 искажённые позиции шифрограмм и отличающиеся от оригинала символы расшифровок выделены полужирным начертанием.

Таблица 3 – Шифрограммы, внесённые искажения и результаты дешифрации

Ключ	Шифрограмма	После вмешательства	Результат дешифрации
K_2	**FV**Z**S**ZQFACSMM	**ZX**Z**C**ZQFACSMM	**PKEV**VE_ITMO!
K_3	**QG**Z**H**LZSJW?JT	**ZX**Z**C**LZSJW?JT	**NVIYXV**_ITMO!
K_4	**TW**!**U**XGRXFRWT	**ZX**!**C**XGRXFRWT	**TFWE**VE_ITMO!

Сводка по числу пострадавших символов приведена в таблице 4.

Таблица 4 – Зависимость числа искажённых символов от размера блока

Ключ	Размер блока	Всего блоков	Задето блоков	Искажено символов
K_2	2	6	2	4
K_3	3	4	2	6
K_4	4	3	1	4

Вывод по заданию 1

Шифрование и дешифрация выполнены для всех трёх ключей. При отсутствии вмешательства обратное преобразование восстанавливает исходное сообщение полностью, то есть метод Хилла работает корректно, а построенные обратные по модулю матрицы найдены верно — это же подтверждается равенством $K K^{-1} \equiv E$.

Основной результат касается распространения ошибок. Искажение шифрограммы не выходит за пределы того блока, в который попал испорченный символ: остальная часть сообщения расшифровывается без потерь. Причина в том, что при шифровании каждый блок умножается на матрицу независимо от других, и информация между блоками не смешивается.

При этом число пострадавших символов определяется не количеством ошибок, а тем, как они распределились по блокам. Три искажения в одних и тех же позициях дали разный результат: 4 испорченных символа для ключа K_2 и столько же для K_4, но 6 символов для K_3. Для K_2 ошибки задели два блока по два символа, для K_3 — тоже два блока, но уже по три символа, а для K_4 все три ошибки пришлись на один четырёхсимвольный блок. Отсюда следует, что с ростом размера блока растёт и цена одиночной ошибки, зато уменьшается вероятность задеть несколько блоков сразу.

Внутри задетого блока портятся, вообще говоря, все символы. Позиция i расшифрованного блока искажается тогда, когда элемент $(K^{-1})_{ij}$ отличен от нуля, где j — позиция испорченного символа. Отсюда видно, при каком условии одна ошибка приводила бы ровно к одной ошибке в расшифровке: в каждой строке обратной матрицы должен стоять ровно один ненулевой элемент. Однако такой ключ обесценивает шифр — каждый символ всего лишь умножается на свою константу и, возможно, меняет позицию, смешивания между символами не происходит. Шифр вырождается в моноалфавитную подстановку, которая вскрывается частотным анализом.

2 Задание 2. Взлом шифра Хилла

2.1 Постановка

Моделируется следующая ситуация: имеются две шифрограммы, полученные шифром Хилла с одним и тем же ключом, и этот ключ неизвестен. Однако в руки попала расшифровка (оригинал) одной из них. Требуется найти способ прочитать вторую.

2.2 Исходные данные

Используется алфавит из задания 1 ($n = 29$). Ключ размера 2×2 генерируется программой случайно: каждый элемент берётся из диапазона $[0, 28]$, после чего проверяется условие $\det K \not\equiv 0 \pmod{29}$, и при его нарушении матрица генерируется заново. Значение ключа при выводе не используется — он известен только программе. Для воспроизводимости расчётов зафиксировано начальное значение генератора: `random.seed(67)`.

Взяты два сообщения по 12 символов, `PRETTY_GIRLS` и `STAY_WITH_ME`, с числовыми представлениями

$$\begin{aligned} m_1 &= (15,\ 17,\ 4,\ 19,\ 19,\ 24,\ 26,\ 6,\ 8,\ 17,\ 11,\ 18), \\ m_2 &= (18,\ 19,\ 0,\ 24,\ 26,\ 22,\ 8,\ 19,\ 7,\ 26,\ 12,\ 4). \end{aligned} \tag{15}$$

Шифрование обоих сообщений неизвестным ключом даёт шифрограммы `XBHYXOMGJHSF` и `GMOWCLPEFNHV`.

После этого первое сообщение считается забытым (в программе оно заменено на строку из двенадцати символов `?`). Атакующему доступны: обе шифрограммы и открытый текст второго сообщения. Ключ недоступен.

2.3 Вывод способа нахождения ключа

Дешифрация одного блока записывается как

$$D x \equiv b \pmod{n}, \tag{16}$$

где D — матрица дешифрования, x — блок шифрограммы, b — соответствующий блок открытого текста. Неизвестной здесь является

именно D, поэтому её оставляем слева, а x переносим вправо, где она обращается.

Одного блока для этого недостаточно. Умножение блока на матрицу 2×2 даёт два уравнения, тогда как в ключе четыре неизвестных элемента — система недоопределена. Поэтому вместо вектора x берётся матрица X, столбцами которой служат два блока шифрограммы, и точно так же из соответствующих блоков открытого текста составляется матрица B. Два блока дают четыре уравнения, чего ровно хватает. Выбранные блоки при этом не должны быть линейно зависимы по модулю n, иначе матрица X необратима и решение не существует. Уравнение (16) превращается в матричное

$$DX \equiv B \pmod{n} \qquad \Longrightarrow \qquad D \equiv BX^{-1} \pmod{n}. \tag{17}$$

Исходный ключ получается ещё одним обращением: $K = D^{-1} \bmod n$.

2.4 Нахождение ключа

Берутся первые два блока. Из шифрограммы второго сообщения составляется

$$X = \begin{pmatrix} 6 & 14 \\ 12 & 22 \end{pmatrix}, \qquad \det X = -36 \equiv 22 \pmod{29}, \tag{18}$$

определитель отличен от нуля, значит блоки линейно независимы и матрица обратима. Обратный элемент к определителю равен $22^{-1} \equiv 4$, откуда

$$X^{-1} \equiv 4 \cdot \begin{pmatrix} 22 & -14 \\ -12 & 6 \end{pmatrix} \equiv \begin{pmatrix} 1 & 2 \\ 10 & 24 \end{pmatrix} \pmod{29}. \tag{19}$$

Из открытого текста второго сообщения составляется матрица тех же двух блоков $B = \begin{pmatrix} 18 & 0 \\ 19 & 24 \end{pmatrix}$, и по формуле (17)

$$D \equiv \begin{pmatrix} 18 & 0 \\ 19 & 24 \end{pmatrix} \begin{pmatrix} 1 & 2 \\ 10 & 24 \end{pmatrix} \equiv \begin{pmatrix} 18 & 7 \\ 27 & 5 \end{pmatrix} \pmod{29}. \tag{20}$$

Обращение найденной матрицы даёт сам ключ:

$$K = D^{-1} \equiv \begin{pmatrix} 2 & 3 \\ 24 & 13 \end{pmatrix} \pmod{29}. \tag{21}$$

2.5 Проверка и расшифровка первого сообщения

Проверка выполняется независимо от того, как ключ был получен: известный открытый текст второго сообщения шифруется *найденным* ключом K, и результат сравнивается с имеющейся шифрограммой. Оба значения совпадают:

$$\texttt{GMOWCLPEFNHV} \quad = \quad \texttt{GMOWCLPEFNHV}. \tag{22}$$

Совпадение не сводится к построению: в выводе ключа участвовали только первые два блока, а сходятся все шесть.

Расшифровка первой шифрограммы найденной матрицей D даёт `PRETTY_GIRLS` — забытое сообщение восстановлено.

Вывод по заданию 2

Шифр Хилла удалось взломать, имея на руках одну шифрограмму вместе с её открытым текстом. Причина в линейности преобразования: если два сообщения зашифрованы одним ключом и рассекречены обе версии одного из них — открытая и зашифрованная, — то пары «блок открытого текста — блок шифрограммы» дают систему линейных уравнений относительно элементов ключа, а обратимость матриц позволяет выразить неизвестную матрицу в явном виде.

Для ключа размера $z \times z$ требуется z^2 символов открытого текста: в ключе z^2 неизвестных, а каждый блок даёт лишь z уравнений. При меньшем объёме известного открытого текста система остаётся недоопределённой и ключ однозначно не восстанавливается.

Отсюда следует, что шифр проваливается при утечке всего z^2 расшифрованных символов: они позволяют раскрыть ключ целиком, а вместе с ним и всю переписку, зашифрованную тем же ключом. По меркам современных шифров это недопустимо — там знание пар «открытый текст — шифрограмма» ключа не выдаёт. Устранить уязвимость можно,

отказавшись от постоянного ключа и используя для каждого сообщения свой.

3 Задание 3. Код Хэмминга

В этом задании решается задача, обратная по смыслу заданиям 1 и 2: не скрыть содержание сообщения, а защитить его от искажений при передаче. Меняются и исходные данные, и арифметика. Алфавит берётся другой — русский, из 32 букв, — и вычисления ведутся не в кольце вычетов $\mathbb{Z}_{29}$, как в первых двух заданиях, а в поле $\mathbb{F}_2$, где сложение совпадает с операцией «исключающее или», то есть $1 + 1 = 0$. Ни алфавит, ни модуль из заданий 1 и 2 далее не используются.

3.1 Алфавит и исходное сообщение

Взят русский алфавит без буквы «Ё», всего 32 буквы. Число $32 = 2^5$, поэтому каждой букве соответствует ровно пятибитовый двоичный номер и ни одна комбинация из пяти бит не пропадает. Буква «Ё» исключена намеренно: с ней алфавит содержал бы 33 буквы, что потребовало бы уже шести бит на букву, и 31 комбинация из 64 осталась бы неиспользованной. Соответствие букв, номеров и двоичных кодов приведено в таблице 5.

Таблица 5 – Соответствие букв алфавита, их номеров и пятибитовых кодов

Буква	А	Б	В	Г	Д	Е	Ж	З
Номер	0	1	2	3	4	5	6	7
Код	00000	00001	00010	00011	00100	00101	00110	00111
Буква	И	Й	К	Л	М	Н	О	П
Номер	8	9	10	11	12	13	14	15
Код	01000	01001	01010	01011	01100	01101	01110	01111
Буква	Р	С	Т	У	Ф	Х	Ц	Ч
Номер	16	17	18	19	20	21	22	23
Код	10000	10001	10010	10011	10100	10101	10110	10111
Буква	Ш	Щ	Ъ	Ы	Ь	Э	Ю	Я
Номер	24	25	26	27	28	29	30	31
Код	11000	11001	11010	11011	11100	11101	11110	11111

Кодируемое слово — «СОНЯ». Его побуквенное двоичное представление приведено в таблице 6.

Таблица 6 – Исходное слово и его двоичное представление

Буква	С	О	Н	Я
Номер	17	14	13	31
Код	10001	01110	01101	11111

Склеивание кодов даёт двоичную последовательность из 20 символов:

$$10001011100110111111. \tag{23}$$

Число 20 делится на 4 нацело, поэтому при разбиении на блоки по четыре бита получается ровно пять блоков и последний не приходится дополнять нулями. В отличие от задания 1, здесь это не результат выбора: длина слова (четыре буквы) задана условием, ширина буквы (пять бит) определяется размером алфавита, а произведение $4 \cdot 5 = 20$ просто оказалось кратно четырём. При слове из трёх букв получилось бы 15 бит, и последний блок пришлось бы дополнять. Существенное же отличие от задания 1 в другом: там размер блока выбирался вместе с размером ключа, здесь он жёстко задан самим кодом — код Хэмминга $(7, 4)$ принимает на вход ровно четыре бита.

3.2 Построение матриц G и H

Если дописать к сообщению один бит чётности, можно установить факт наличия ошибки, но не её положение. Чтобы определить положение, проверок должно быть несколько, и их совокупный результат должен различать позиции. Три проверки дают три бита ответа, то есть 8 различных значений. Одно значение (000) отводится под случай «ошибок нет», остаются 7 — ровно столько позиций и имеет смысл брать в кодовом слове. Отсюда длина 7: большее число позиций три проверки уже не различат, меньшее оставит часть значений неиспользованной. Из семи позиций три занимают сами проверочные биты, четыре остаются информационными, что и даёт обозначение $(7, 4)$.

Позиции кодового слова нумеруются числами от 1 до 7. Под проверочные биты отводятся позиции, являющиеся степенями двойки: 1, 2 и 4; остальные позиции — 3, 5, 6, 7 — занимают информационные биты d_1, d_2, d_3, d_4. Проверочные биты нумеруются не подряд, а по номеру

занимаемой позиции: $p_1,\ p_2,\ p_4$. При таком обозначении всё правило формулируется одной фразой: проверочный бит p_k стоит на позиции k и отвечает за бит веса k в двоичной записи номера позиции. Кодовое слово имеет вид

$$c = (p_1,\ p_2,\ d_1,\ p_4,\ d_2,\ d_3,\ d_4)^{\mathsf{T}}. \tag{24}$$

Проверка p_k охватывает те позиции, в двоичной записи номера которых присутствует бит веса k. Это даёт три группы и три формулы:

$$\begin{aligned} &p_1 \text{ — позиции } 1,3,5,7: && p_1 = d_1 + d_2 + d_4, \\ &p_2 \text{ — позиции } 2,3,6,7: && p_2 = d_1 + d_3 + d_4, \\ &p_4 \text{ — позиции } 4,5,6,7: && p_4 = d_2 + d_3 + d_4. \end{aligned} \tag{25}$$

Все суммы берутся по модулю 2. Каждая проверка настроена так, чтобы сумма всех охваченных ею битов, включая сам проверочный, равнялась нулю.

Выбор степеней двойки под проверочные биты объясняется именно этим: позиция 1 входит только в первую группу, позиция 2 — только во вторую, позиция 4 — только в третью. Поэтому каждый проверочный бит вычисляется непосредственно из информационных и не входит в чужие уравнения. При размещении проверочных битов на любых других позициях они оказались бы связаны друг с другом, и систему пришлось бы решать отдельно.

Порождающая матрица G представляет собой запись формул (25) и переноса информационных битов по строкам; столбцов в ней четыре — по числу информационных битов. Кодирование выполняется умножением

$$c = Gd \bmod 2, \tag{26}$$

$$G = \begin{pmatrix} 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}. \tag{27}$$

Строки сверху вниз соответствуют позициям p_1, p_2, d_1, p_4, d_2, d_3, d_4. Строки 3, 5, 6 и 7 — это строки единичной матрицы, они переносят информационные биты без изменений; строки 1, 2 и 4 содержат коэффициенты трёх проверок.

Проверочная матрица H строится ещё проще: в столбец с номером j записывается двоичное представление числа j, старший бит сверху:

$$H = \begin{pmatrix} 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 \end{pmatrix}. \tag{28}$$

Её столбцы — это числа $001, 010, 011, 100, 101, 110, 111$, то есть все числа от 1 до 7, и каждый ненулевой вектор пространства $\mathbb{F}_2^3$ встречается ровно один раз. Строки при этом автоматически оказываются теми самыми проверками: в нижней строке единицы стоят на позициях 1, 3, 5, 7 (номера, содержащие бит веса 1), в средней — на позициях 2, 3, 6, 7, в верхней — на позициях 4, 5, 6, 7. Таким образом, H есть матричная запись трёх уравнений проверки, а условие $Hc = 0$ означает, что все три проверки сошлись.

Порядок строк H выбран так, что сверху стоит бит веса 4, а снизу — бит веса 1. Благодаря этому синдром, прочитанный сверху вниз как двоичное число, сразу даёт номер испорченной позиции, и никакой таблицы соответствия не требуется.

Согласованность матриц проверяется равенством

$$HG \equiv 0 \pmod 2, \tag{29}$$

где справа стоит нулевая матрица размера 3×4; вычисление выполнено программно (см. листинг 4). Это равенство и означает, что любое кодовое слово $c = Gd$ проходит все три проверки.

3.3 Кодирование сообщения

Последовательность (23) разбивается на пять блоков по четыре бита: `1000`, `1011`, `1001`, `1011`, `1111`. Каждый блок рассматривается как вектор-столбец d и умножается на матрицу G по формуле (26). Покажем расчёт на

третьем блоке $d = (1, 0, 0, 1)$:

$$Gd = \begin{pmatrix} 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \\ 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 1+0+0+1 \\ 1+0+0+1 \\ 1+0+0+0 \\ 0+0+0+1 \\ 0+0+0+0 \\ 0+0+0+0 \\ 0+0+0+1 \end{pmatrix} = \begin{pmatrix} 2 \\ 2 \\ 1 \\ 1 \\ 0 \\ 0 \\ 1 \end{pmatrix} \equiv \begin{pmatrix} 0 \\ 0 \\ 1 \\ 1 \\ 0 \\ 0 \\ 1 \end{pmatrix} \pmod 2, \tag{30}$$

то есть кодовое слово третьего блока равно `0011001`. Проверочные биты при этом получаются равными $p_1 = d_1 + d_2 + d_4 = 1 + 0 + 1 \equiv 0$, $p_2 = d_1 + d_3 + d_4 = 1 + 0 + 1 \equiv 0$ и $p_4 = d_2 + d_3 + d_4 = 0 + 0 + 1 = 1$, что согласуется с формулами (25). Остальные блоки обработаны по той же схеме программно; результаты приведены в таблице 7.

Таблица 7 – Кодирование блоков сообщения матрицей G

Блок	Информационные биты	Кодовое слово
1	`1000`	`1110000`
2	`1011`	`0110011`
3	`1001`	`0011001`
4	`1011`	`0110011`
5	`1111`	`1111111`

Закодированное сообщение длиной 35 бит:

$$\texttt{1110000 0110011 0011001 0110011 1111111}. \tag{31}$$

Длина выросла с 20 до 35 бит, то есть в $7/4$ раза. Это и есть плата за возможность исправлять ошибки: три дополнительных бита на каждые четыре информационных.

3.4 Вредоносное вмешательство

Биты закодированного сообщения нумеруются с единицы сквозной нумерацией по всем 35 позициям; блок с номером b занимает позиции с $7(b-1)+1$ по $7b$.

Проведены две серии по четыре опыта. В каждой серии последовательно искажаются 1, 2, 3 и 4 бита, причём каждый следующий опыт добавляет ошибку к предыдущим. Серии отличаются только распределением ошибок по блокам, и именно это различие является предметом исследования.

Серия A. Все ошибки накапливаются внутри одного блока — третьего, занимающего позиции 15–21. Искажаются позиции 17, затем 17 и 19, затем 17, 19 и 20, затем 17, 19, 20 и 21; внутри блока это локальные позиции 3, 5, 6 и 7. Закодированное сообщение с выделенными искажаемыми битами:

$$1110000\,0110011\,00\mathbf{11}\mathbf{00}\mathbf{1}\,0110011\,1111111. \tag{32}$$

Соответствующие изменения третьего блока приведены в таблице 8.

Таблица 8 – Серия A: искажение битов внутри одного блока

Опыт	Глобальные позиции	Локальные позиции	Третий блок
1 бит	17	3	`0011001` → `0001001`
2 бита	17, 19	3, 5	`0011001` → `0001101`
3 бита	17, 19, 20	3, 5, 6	`0011001` → `0001111`
4 бита	17, 19, 20, 21	3, 5, 6, 7	`0011001` → `0001110`

Серия B. Столько же ошибок, но каждая очередная вносится в новый блок. Искажаются позиции 3, 12, 22 и 35:

$$11\mathbf{1}0000\,0110\mathbf{0}11\,0011001\,\mathbf{0}110011\,1111111\mathbf{1}. \tag{33}$$

Распределение ошибок по блокам приведено в таблице 9. Третья ошибка намеренно внесена не в информационный, а в проверочный бит — чтобы проверить, как код ведёт себя при повреждении собственной служебной части.

Таблица 9 – Серия B: распределение ошибок по разным блокам

Опыт	Позиция	Блок	Локальная позиция	Бит	Задето блоков
1 бит	3	1	3	d_1, информационный	1
2 бита	12	2	5	d_2, информационный	2
3 бита	22	4	1	p_1, проверочный	3
4 бита	35	5	7	d_4, информационный	4

3.5 Декодирование по синдрому

Принятый блок представляется как $r = c + e$, где c — переданное кодовое слово, а e — вектор ошибок, содержащий единицы на тех позициях, где биты изменились. Синдром вычисляется как

$$s = Hr = H(c + e) = Hc + He = He \pmod 2, \tag{34}$$

поскольку $Hc = 0$ по построению. В этом и состоит основное свойство метода: синдром не зависит от передаваемого сообщения и определяется исключительно вектором ошибок. Декодеру не требуется знать, что именно было отправлено.

Если ошибка ровно одна и находится на позиции j, то e — столбец единичной матрицы, и произведение He равно j-му столбцу матрицы H, который по построению совпадает с двоичной записью числа j. Прочитанный сверху вниз синдром непосредственно даёт номер испорченного бита, а значение $s = (0, 0, 0)$ означает, что все проверки сошлись и исправлять нечего.

После исправления информационные биты извлекаются с позиций 3, 5, 6 и 7. В программе это выполнено умножением на матрицу выборки

$$R = \begin{pmatrix} 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{pmatrix}, \qquad RG \equiv E \pmod 2. \tag{35}$$

Равенство $RG = E$ проверено программно; оно означает, что из неискажённого кодового слова исходный блок извлекается в точности в том виде, в каком он в него вошёл. Полученные 20 бит разбиваются по пять и переводятся в буквы по таблице 5.

Результаты серии А приведены в таблице 10.

Таблица 10 – Серия A: синдромы, исправленный код и результат декодирования

Опыт	Синдром	Указанный бит	Третий блок после исправления	Слово
1 бит	011	3	0011001	СОНЯ
2 бита	110	6	0001111	СНЭЯ
3 бита	000	ошибок нет	0001111	СНЭЯ
4 бита	111	7	0001111	СНЭЯ

Декодированные последовательности: в первом опыте 10001011100110111111, то есть исходное сообщение (23); в остальных трёх — 10001011011110111111. У четырёх остальных блоков синдром во всех опытах серии A равен 000, после исправления они остаются неизменными (1110000, 0110011, 0110011, 1111111) и декодируются без потерь. В колонке исправленного кода видно то, что подробно разбирается ниже: три неудачных опыта приходят в одну и ту же точку 0001111.

Результаты серии B приведены в таблице 11.

Таблица 11 – Серия B: синдромы по блокам и результат декодирования

Опыт	Синдромы блоков 1–5	Код после исправления	Слово
1 бит	011, 000, 000, 000, 000	совпадает с (31)	СОНЯ
2 бита	011, 101, 000, 000, 000	совпадает с (31)	СОНЯ
3 бита	011, 101, 000, 001, 000	совпадает с (31)	СОНЯ
4 бита	011, 101, 000, 001, 111	совпадает с (31)	СОНЯ

Совпадение с (31) проверено побитово: восстанавливаются все пять блоков целиком, а не только правильные буквы на выходе. Таким образом, в серии B все четыре опыта, включая опыт с четырьмя искажёнными битами, восстановили исходное слово полностью. Во всех опытах декодированная последовательность равна (23).

Отдельного разбора заслуживает третий опыт серии B, в котором искажён проверочный бит p_1 четвёртого блока. Синдром оказался равен $001 = 1$, декодер указал на первый бит и исправил его точно так же, как исправлял информационные. Это отвечает на вопрос, зачем проверять сами проверочные биты: декодер заранее не знает, куда попала ошибка, для него все семь позиций равноправны, и матрица H не делает между ними никакого различия — в её столбцах стоят просто номера от 1 до 7. Одновременно

видно, что ошибка в проверочном бите не угрожает данным: даже если оставить её на месте, информационные биты извлекаются с позиций 3, 5, 6, 7 и не пострадают. Однако установить, что ошибка находится именно в проверочном бите, можно лишь после вычисления синдрома, поэтому исправление выполняется в любом случае.

3.6 Разбор результатов серии A

Одна ошибка (позиция 17, локальная 3). Синдром равен $011 = 3$, что в точности совпадает с номером искажённого бита. Декодер переворачивает его обратно и получает исходное кодовое слово; слово «СОНЯ» восстановлено полностью.

Две ошибки (локальные позиции 3 и 5). Синдром равен сумме двух столбцов матрицы H:

$$s = h_3 + h_5 = 011 + 101 = 110 = 6. \tag{36}$$

Декодер трактует это как одиночную ошибку в шестом бите и переворачивает шестой бит. В результате в блоке оказывается три искажённых бита — 3, 5 и 6 — вместо двух: декодер не только не исправил ошибку, но и добавил третью. Существенно здесь не то, что не было подано сигнала тревоги, а то, чем декодер это обосновал: на выходе у него блок `0001111`, а это законное кодовое слово, поскольку $\texttt{0001111} = G\,(0,1,1,1)^{\mathsf{T}}$, синдром у него нулевой, и любая повторная проверка его пропустит. Отличить такой результат от истинного невозможно не из-за несовершенства декодера, а потому, что необходимой для этого информации в принятом блоке уже не осталось: получилось слово, которое имело полное право быть переданным.

Три ошибки (локальные позиции 3, 5, 6). Синдром равен 000, декодер сообщает об отсутствии ошибок и выдаёт данные как есть. Это наихудший случай — ошибка остаётся незамеченной. Причина в том, что сам набор позиций $\{3, 5, 6\}$ образует кодовое слово: $e = \texttt{0010110} = G\,(1,1,1,0)^{\mathsf{T}}$. Сумма двух кодовых слов снова является кодовым словом, а на кодовых словах синдром равен нулю по определению. Три ошибки перевели принятый вектор точно в соседнее законное слово.

Четыре ошибки (локальные позиции 3, 5, 6, 7). Синдром равен $h_3 + h_5 + h_6 + h_7 = 111 = 7$, декодер переворачивает седьмой бит и

тем самым устраняет четвёртую ошибку, оставаясь при том же наборе $\{3, 5, 6\}$, что и в предыдущем опыте. Поэтому результат побитово совпадает с трёхошибочным.

Во всех трёх неудачных опытах серии А получено одно и то же слово «СНЭЯ», и это не случайность: как видно из таблицы 10, все три случая сходятся в одно кодовое слово `0001111`. Декодер по построению всегда выдаёт какое-либо кодовое слово — других вариантов у него нет, — и при числе ошибок больше одной он выдаёт не переданное слово, а ближайшее к принятому. В опытах 2, 3 и 4 ближайшим каждый раз оказалось `0001111`, откуда и одинаковый ответ.

Отдельно следует объяснить, почему пострадали именно буквы «О» и «Н». Третий блок кодирует биты 9–12 двадцатибитной последовательности, тогда как вторая буква занимает биты 6–10, а третья — биты 11–15. Четырёхбитный блок лёг на стык двух букв: биты 9 и 10 относятся ко второй букве, биты 11 и 12 — к третьей. Поэтому искажение одного блока из четырёх бит испортило сразу две буквы из четырёх: О $\to$ Н и Н $\to$ Э.

Это частный случай общего соотношения. Блок кода составляет 4 бита, буква — 5 бит, а числа 4 и 5 взаимно просты, поэтому их границы совпадают только через $\operatorname{lcm}(4, 5) = 20$ бит, то есть ровно один раз на всё сообщение. Из пяти блоков целиком внутри одной буквы лежат только первый (биты 1–4 при букве, занимающей биты 1–5) и пятый (биты 17–20 при букве, занимающей биты 16–20); три средних блока расположены на стыке и захватывают по две буквы каждый.

Однако покрытие задаёт лишь верхнюю границу ущерба, а не сам ущерб: число реально испорченных букв зависит ещё и от того, какие биты внутри блока изменились после ложного исправления. В рассмотренных опытах ошибки в локальных позициях 3 и 5 после исправления дают набор $\{3, 5, 6\}$, то есть переворачивают биты d_1, d_2, d_3 и не затрагивают d_4: последний бит блока каждый раз остаётся неизменным. Результаты применения того же набора ошибок к каждому из пяти блоков приведены в таблице 12.

Таблица 12 – Число испорченных букв при одном и том же наборе ошибок в разных блоках

Испорченный блок	Покрываемые буквы	Испорчено букв	Слово
1	1	1	НОНЯ
2	1, 2	2	РЦНЯ
3	2, 3	2	СНЭЯ
4	3, 4	1	СОКЯ
5	4	1	СОНС

Четвёртый блок выбивается из общей картины: стык он пересекает, но изменяются у него биты 13, 14 и 15 — все три внутри третьей буквы, — тогда как единственный его бит, принадлежащий четвёртой букве, это бит 16, то есть уцелевший d_4. Следовательно, две буквы портятся не всегда, когда блок расположен на стыке, а только когда граница букв проходит внутри тех его битов, которые действительно изменились.

Вывод по заданию 3

Код Хэмминга $(7, 4)$ построен, слово «СОНЯ» закодировано в 35 бит и восстановлено после вмешательства. Обе матрицы согласованы: равенства $HG = 0$ и $RG = E$ проверены прямым умножением.

Основной результат состоит в том, что код исправляет одну ошибку *на блок*, а не одну ошибку на сообщение. Обе серии опытов отличались только распределением ошибок по блокам, а результаты получились противоположными. Четыре ошибки, разнесённые по четырём разным блокам (серия В), исправлены все до одной, и слово восстановлено полностью. Две ошибки, попавшие в один блок (серия А), код уже не исправил. Следовательно, характеристика «код исправляет одну ошибку» относится к блоку из семи бит, и на 35-битном сообщении из пяти блоков предел составляет пять исправляемых ошибок — но лишь при условии, что они распределены по одной на блок.

Это та же логика блочности, что и в задании 1: там искажение шифрограммы не выходило за пределы своего блока, здесь ошибка исправляется или не исправляется независимо от того, что происходит в соседних блоках. Различие в том, что в шифре Хилла размер блока определялся выбранным размером ключа, а здесь он задан самим кодом.

Причина, по которой две ошибки не исправляются, видна из равенства (34). Синдром есть сумма столбцов матрицы H на позициях ошибок, и принимает он всего 8 значений, тогда как возможных векторов ошибок $2^7 = 128$. Однозначно распознаются лишь 8 из них: нулевой и семь одиночных. Любой другой вектор ошибок даёт синдром, совпадающий с синдромом какой-либо одиночной ошибки, и декодер выбирает именно её — как наиболее правдоподобный вариант из всех, дающих такой синдром. Разложение $s = He$ показывает и то, почему декодирование вообще возможно без знания исходного сообщения: синдром зависит только от ошибки.

Отдельно следует назвать наиболее неблагоприятный сценарий — три ошибки с нулевым синдромом. В этом случае код не просто ошибается, но и не обнаруживает ошибку вовсе, выдавая результат как достоверный. Происходит это тогда, когда вектор ошибок сам оказывается кодовым словом. Минимальный вес ненулевого кодового слова в коде Хэмминга $(7, 4)$ равен трём (таких слов семь), откуда минимальное кодовое расстояние $d = 3$. Общее правило: код исправляет $\lfloor (d-1)/2 \rfloor = 1$ ошибку и обнаруживает $d - 1 = 2$ ошибки. Проведённый опыт с двумя ошибками — как раз случай, когда ошибка обнаружима (синдром ненулевой), но неисправима: декодер видит несоответствие, но исправляет не ту позицию. Различать эти два режима код самостоятельно не умеет; чтобы он только обнаруживал ошибки и не портил данные, его требуется принудительно переводить в режим обнаружения.

Наконец, о контрольном вопросе, касающемся пространств. Столбцы матрицы G порождают линейную оболочку $\operatorname{Im} G \subset \mathbb{F}_2^7$ — множество всех 16 кодовых слов размерности 4, поскольку $\operatorname{rank} G = 4$: строки 3, 5, 6 и 7 матрицы G образуют единичную матрицу 4×4, значит её столбцы линейно независимы. Ядро $\operatorname{Ker} H$ — множество векторов, проходящих все три проверки, и его размерность равна $7 - \operatorname{rank} H$. Ранг H равен трём, что также видно непосредственно из построения: столбцы 1, 2 и 4 матрицы H — это 001, 010 и 100, вместе они составляют единичную матрицу 3×3, поэтому три строки H линейно независимы, а большего числа строк в ней нет. Отсюда $\dim \operatorname{Ker} H = 7 - 3 = 4$.

Оба ранга вычисляются над полем $\mathbb{F}_2$, и это требует отдельной оговорки. Здесь ответ над $\mathbb{Q}$ случайно совпал бы, но совпадать он не

обязан: строки $(1,1,0)$, $(0,1,1)$, $(1,0,1)$ над $\mathbb{Q}$ линейно независимы (их определитель равен 2), тогда как над $\mathbb{F}_2$ их сумма равна нулю и ранг падает до двух. Ранг есть свойство матрицы вместе с полем, а не матрицы самой по себе. Поскольку $HG = 0$, справедливо включение $\operatorname{Im} G \subseteq \operatorname{Ker} H$, а совпадение размерностей даёт равенство

$$\operatorname{Im} G = \operatorname{Ker} H. \tag{37}$$

Матрицы G и H описывают одно и то же множество с двух сторон: G его строит, H его распознаёт. Этим и объясняется, почему кодирование и проверка согласуются автоматически и почему для подтверждения правильности матриц достаточно одной проверки $HG = 0$.

Заключение

В работе матричное умножение применено к двум различным задачам, и обе решались не над полем вещественных чисел, а над конечными структурами: кольцом вычетов $\mathbb{Z}_{29}$ в заданиях 1 и 2 и полем $\mathbb{F}_2$ в задании 3. Это оказалось не формальностью, а обстоятельством, определяющим всю технику работы. Обратимость матрицы перестала сводиться к условию $\det \neq 0$ и превратилась в условие $\gcd(\det K, n) = 1$; деление заменилось умножением на обратный элемент, который приходится искать расширенным алгоритмом Евклида; стандартная функция `numpy.linalg.inv` оказалась неприменимой. Выбор простого модуля $n = 29$ в первых двух заданиях был сделан именно для того, чтобы условие обратимости упростилось до $\det K \not\equiv 0$.

Все три задания оказались посвящены одному свойству — блочности — и тому, чем за неё приходится платить. Матрица применяется к блоку целиком и независимо от соседних блоков. В заданиях 1 и 2 это означает, что искажение не распространяется по сообщению, но внутри своего блока портит, вообще говоря, всё. В задании 3 — что ошибки учитываются и исправляются поблочно, и четыре ошибки в четырёх блоках исправляются, а две ошибки в одном блоке уже нет.

Сравнение двух методов показывает, что они устроены зеркально, и всё различие между ними — это различие между биекцией и вложением. Шифр

Хилла биективен: матрица K квадратная и обратимая, она отображает $\mathbb{Z}_{29}^{z}$ на себя целиком, длина сообщения не меняется, информация не добавляется и не теряется. Код Хэмминга инъективен, но не сюръективен: матрица G прямоугольная и вкладывает $\mathbb{F}_2^4$ в $\mathbb{F}_2^7$ без малейшей потери — именно это означает проверенное равенство $RG = E$, исходный блок извлекается из кодового слова всегда и точно, — однако образ этого вложения занимает лишь 16 точек из 128, то есть на всё пространство G не отображает.

Из этого различия вытекают оба результата работы. У шифра Хилла образ — всё пространство, поэтому любая испорченная шифрограмма остаётся законной шифрограммой, и обнаружить вмешательство невозможно в принципе: в задании 1 искажённые сообщения расшифровались без каких-либо признаков порчи, просто в бессмысленный текст. Та же биективность вместе с линейностью даёт и атаку из задания 2: ключ вычисляется из пары «открытый текст — шифрограмма» по z^2 известным символам. У кода Хэмминга наоборот: образ мал, и само попадание вне образа уже служит сигналом об ошибке. Свободное место вокруг кодовых слов и есть тот ресурс, за счёт которого ошибку удаётся не только заметить, но и локализовать; платой за него служит рост длины сообщения в $7/4$ раза. Отсюда же виден и предел метода: свободного места ровно столько, сколько необходимо на одну ошибку в блоке, а три ошибки переносят принятый вектор в соседнее кодовое слово, то есть обратно внутрь образа, где сигнала нет и быть не может. Шифр защищает от чтения и совершенно не защищает от искажений; код защищает от искажений и ничего не скрывает.

Работа с синдромом показала, ради чего вообще строятся такие матрицы. Равенство $s = H(c + e) = He$ означает, что результат проверки не зависит от передаваемых данных: декодер работает с ошибкой напрямую, не зная и не восстанавливая исходное сообщение. Ограничение метода вытекает из того же равенства: синдром принимает 8 значений при 128 возможных векторах ошибок, поэтому различимы только нулевой вектор и семь одиночных ошибок. Все остальные случаи декодер вынужден трактовать как ближайшую одиночную ошибку, а иногда — когда вектор ошибок сам оказывается кодовым словом — не замечает ошибку вовсе.

Приложение А. Код

```
import string, sympy as sp


alphabet = list(string.ascii_uppercase+'_!?') ### алфавит из n символов (пункт 1)
n = len(alphabet)

def extended_gcd(a, b):
    if b == 0:
        return a, 1, 0
    d, x1, y1 = extended_gcd(b, a % b)
    x = y1
    y = x1 - y1 * (a // b)
    return d, x, y

def to_digits(message):
    return [alphabet.index(i) for i in message]

def to_word(message):
    return ''.join([alphabet[i] for x in message for i in x])

def shifr(message,matrix):
    z = matrix.shape[0]
    splitted_message_in_digits = []
    shifred_message_in_digits = []
    for x in range(len(message)//z):
        splitted_message_in_digits.append([message[i] for i in range(x*z,x*z+z)])

    for item in splitted_message_in_digits:
        shifred_message_in_digits.append((matrix*sp.Matrix(item)%n).flat())

    return shifred_message_in_digits

def deshifr(message,matrix):
    z = matrix.shape[0]
    shifred_message_in_digits = []
    deshifred_message_digits = []
    for x in range(len(message)//z):
```

```
            shifred_message_in_digits.append([message[i] for i in range(x*z, x*z+z)])


    for item in shifred_message_in_digits:
        deshifred_message_digits.append((matrix*sp.Matrix(item)%n).flat())

    return deshifred_message_digits
```

Листинг 1 – Общий модуль `hill.py`

```
import sympy as sp

from hill import extended_gcd, to_digits, to_word, shifr, deshifr, alphabet,
    n

message = 'I_LOVE_ITMO!' ### придумываем ценное сообщение (пункт 2)

m2 = sp.Matrix([[1,1],[2,8]])
m3 = sp.Matrix([[7, 0, 28],[4,4,4],[8,1,2]])
m4 = sp.Matrix([[3,3,17,18],[22,9,1,15],[10,7,8,8],[20,11,7,20]])

mats = [m2,m3,m4] ### придумываем 3 матрицы ключа (пункт 3)

det_m2 = m2.det()
det_m3 = m3.det()
det_m4 = m4.det()


key_m2 = m2.adjugate()*extended_gcd(det_m2,n)[1]%n
key_m3 = m3.adjugate()*extended_gcd(det_m3,n)[1]%n
key_m4 = m4.adjugate()*extended_gcd(det_m4,n)[1]%n

keys = [key_m2,key_m3,key_m4]


### проверка обратности матриц
for i in range(len(mats)):
    if mats[i]*keys[i]%n == sp.Matrix.eye(mats[i].rows,mats[i].cols):
        print(f"матрицы {mats[i]} и {keys[i]} обратны")


```

```
for i in range(len(mats)):
    start_shifr = shifr(to_digits(message),mats[i]) ### шифруем сообщение (пункт 4)
    temp_word = to_word(start_shifr)
    print(f"зашифрованное собщение до подмены символов: {temp_word}")
    ### подмена символов в слове (пункт 5)
    h,j,k = 0,1,3
    temp_word = temp_word[:h] + 'Z' + temp_word[h+1:]
    temp_word = temp_word[:j] + 'X' + temp_word[j+1:]
    temp_word = temp_word[:k] + 'C' + temp_word[k+1:]

    print(f"зашифрованное собщение после подмены символов: {temp_word}")
    end_deshifr = deshifr(to_digits(temp_word),keys[i]) ### дешифруем слово со внедренными символами по индексам h,j,k (пункт 6)
    print(f"расшифрованное сообщение с подменой символов: {to_word(end_deshifr)}")
```

Листинг 2 – Задание 1. Шифр Хилла

```
import sympy as sp
import random


from hill import extended_gcd, to_digits, to_word, shifr, deshifr, alphabet, n

random.seed(67) ### сид для рандома чтобы было легко проверять

m2 = sp.Matrix([[random.randint(0,28),random.randint(0,28)],[random.randint(0,28),random.randint(0,28)]])

while m2.det()%n==0:
    m2 = sp.Matrix([[random.randint(0,28),random.randint(0,28)],[random.randint(0,28),random.randint(0,28)]])

key_m2 = m2.adjugate()*extended_gcd(m2.det(),n)[1]%n

message = 'PRETTY_GIRLS' ### original
message2 = 'STAY_WITH_ME'



```

```
21
22 shifrd_wrd_digits = shifr(to_digits(message),m2)
23 sfrd_msg = to_word(shifrd_wrd_digits)
24 print(f'зашифрованное сообщение 1: {sfrd_msg}')
25
26 message_forgotten = '?'*12 ### я забыл'' сообщение
27
28 shifrd_wrd_digits2 = shifr(to_digits(message2),m2)
29 sfrd_msg_2 = to_word(shifrd_wrd_digits2)
30 print(f'зашифрованное сообщение 2: {sfrd_msg_2}')
31
32
33 print(f'что мы знаем: message1 = {message_forgotten}, message2 = {message2}'
       )
34
35 ### решение
36 ### мы имеем лишь message2, sfrd_msg_2 (шифрованное сообщение 2), sfrd_msg (
       шифрованное сообщение 1), а так же их представления в числах
37
38 ### Dx=b, где D - матрица дешиврования (также известная как found_key_m2),
       обратная матрице m2 по модулю n, x - матрица созданная из первых двух
       векторов в виде столбцов зашифрованного соообщения 2, b - матрица
       созданная из первых двух векторов в виде столбцов расшифрованного
       сообщения 2
39
40
41 x_matrix = sp.Matrix([shifrd_wrd_digits2[0],shifrd_wrd_digits2[1]]).T ###
       матрица созданная из первых двух векторов в виде столбцов зашифрованного
       соообщения 2
42 if x_matrix.det()%n == 0:
43     raise ValueError(f"выберите другие блоки, эти линейно зависимы по
       модулю {n}")
44
45 b_matrix = sp.Matrix([to_digits(message2)[:2],to_digits(message2)[2:4]]).T #
       ## матрица созданная из первых двух векторов в виде столбцов
       расшифрованного сообщения 2
46 x_matrix_inv = x_matrix.adjugate()*extended_gcd(x_matrix.det(),n)[1]%n ###
       матрица обратная
47
48
49 found_key_m2 = b_matrix*x_matrix_inv%n
```

```
K = found_key_m2.adjugate()*extended_gcd(found_key_m2.det(),n)[1]%n
print(f'найденная матрица ключ: {K}, обратная матрица: {found_key_m2}')

print(f'проверяем независимой проверкой.\шифруемn сообщение 2 с помощью
    полученного ключа: {to_word(shifr(to_digits(message2),K))}\зашифрованноеn
     с помощью исходного ключа сообщение 2: {sfrd_msg_2}') ### проверка
    второго сообщения

message_forgotten = to_word(deshifr(to_digits(sfrd_msg),found_key_m2))
print(f'расшифованное сообщение 1: {message_forgotten}')
```

Листинг 3 – Задание 2. Взлом шифра Хилла

```
import sympy as sp

abet = "АБВГДЕЖЗИЙКЛМНОПРСТУФХЦЧШЩЪЫЬЭЮЯ" ### русский алфавит из 32 букв, Ё
    выкинута (пункт 1)
word = "СОНЯ" ### интересное слово из 4 букв (пункт 2)

### порождающая и проверочная матрицы кода Хэмминга (7,4) (пункт 3)
### позиции 1,2,4 (степени двойки) отданы под проверочные биты p1,p2,p4,
    позиции 3,5,6,7 – под информационные
### проверочный бит pk стоит на позиции k и отвечает за бит веса k в номере
    позиции
### строки G это формулы для каждого бита кодового слова: c = G*d
G = sp.Matrix([[1,1,0,1],   ### p1 = d1+d2+d4
               [1,0,1,1],   ### p2 = d1+d3+d4
               [1,0,0,0],   ### d1
               [0,1,1,1],   ### p4 = d2+d3+d4
               [0,1,0,0],   ### d2
               [0,0,1,0],   ### d3
               [0,0,0,1]])  ### d4

### столбец j матрицы H это двоичная запись числа j, поэтому синдром сразу
    дает номер испорченного бита
H = sp.Matrix([[0,0,0,1,1,1,1],
               [0,1,1,0,0,1,1],
               [1,0,1,0,1,0,1]])

### матрица выборки информационных битов из кодового слова (позиции 3,5,6,7)
R = sp.Matrix([[0,0,1,0,0,0,0],
               [0,0,0,0,1,0,0],
```

```
                [0,0,0,0,0,1,0],
                [0,0,0,0,0,0,1]])


def to_binary(x):
    return f'{x:05b}'

def to_bits(message):
    return [int(i) for i in ''.join(to_binary(abet.index(x)) for x in
   message)]

def to_word(bits):
    s = ''.join(str(i) for i in bits)
    return ''.join(abet[int(s[i:i + 5], 2)] for i in range(0, len(s), 5))

def to_str(bits):
    return ''.join(str(i) for i in bits)

def split(bits, z):
    return [bits[i:i + z] for i in range(0, len(bits), z)]

def code(bits): ### кодирование, каждый блок из 4 битов превращается в 7
   битов (пункт 4)
    coded = []
    for block in split(bits, 4):
        coded += [int(i) for i in (G * sp.Matrix(block) % 2)]
    return coded

def spoil(bits, positions): ### вредоносное вмешательство, биты нумеруются
   с единицы (пункт 5)
    spoiled = bits[:]
    for p in positions:
        spoiled[p - 1] ^= 1
    return spoiled

def where(p): ### в каком блоке и на какой локальной позиции стоит бит p
   сквозной нумерации
    return (p - 1) // 7 + 1, (p - 1) % 7 + 1

def kind(p): ### проверочный бит или информационный, по локальной позиции
   внутри блока
```

```
    names = {1: 'p1', 2: 'p2', 4: 'p4', 3: 'd1', 5: 'd2', 6: 'd3', 7: 'd4'}
    return names[where(p)[1]]

def decode(bits): ### декодирование с поиском и исправлением ошибки по синдрому (пункт 6)
    decoded = []
    syndromes = []
    fixed = []
    for block in split(bits, 7):
        block = block[:]
        s = [int(i) for i in (H * sp.Matrix(block) % 2)]
        pos = s[0] * 4 + s[1] * 2 + s[2] ### синдром это двоичная запись номера испорченного бита
        if pos != 0:
            block[pos - 1] ^= 1
        syndromes.append((to_str(s), pos))
        fixed.append(to_str(block)) ### блок после исправления, до выборки информационных битов
        decoded += [int(i) for i in (R * sp.Matrix(block) % 2)]
    return decoded, syndromes, fixed


### таблица соответствий букв и их пятибитовых номеров (пункт 1)
print("таблица алфавита:")
for i in range(len(abet)):
    print(f"{abet[i]} = {i} = {to_binary(i)}", end='  ' if (i + 1) % 8 else '\n')

### проверка того что H*G = 0, то есть каждое кодовое слово лежит в ядре H
if (H * G % 2) == sp.zeros(3, 4):
    print(f"H*G = 0, образ G лежит в ядре H")
### проверка того что R*G = E, то есть выборка информационных битов возвращает исходный блок
if (R * G % 2) == sp.Matrix.eye(4):
    print(f"R*G = E, информационные биты достаются из кодового слова без изменений")

bits = to_bits(word)
print(f"слово {word} в двоичном коде: {to_str(bits)} ({len(bits)} бит)")
print(f"блоки по 4 бита: {[to_str(b) for b in split(bits, 4)]}")

```

```
coded = code(bits)
print(f"закодированное сообщение: {to_str(coded)} ({len(coded)} бит)")
print(f"блоки по 7 бит: {[to_str(b) for b in split(coded, 7)]}")

### серия A: ошибки копятся внутри одного блока (третьего, биты 15–21)
### серия B: столько же ошибок, но разнесенных по разным блокам
series = {"A (все ошибки в третьем блоке)": [[17], [17, 19], [17, 19, 20], [17, 19, 20, 21]],
          "B (ошибки в разных блоках)": [[3], [3, 12], [3, 12, 22], [3, 12, 22, 35]]}

for name in series:
    print(f"\серия {name}")
    for positions in series[name]:
        spoiled = spoil(coded, positions)
        decoded, syndromes, fixed = decode(spoiled)
        print(f"  портим {len(positions)} бит на позициях {positions}")
        print(f"    это (блок, локальная позиция, бит): {[(*where(p), kind(p)) for p in positions]}")
        print(f"    испорченный код:  {[to_str(b) for b in split(spoiled, 7)]}")
        print(f"    синдромы (двоичный, номер бита): {syndromes}")
        print(f"    исправленный код: {fixed}" + ("" if fixed == [to_str(b) for b in split(coded, 7)] else "  <- отличается от исходного"))
        print(f"    декодированные биты: {to_str(decoded)}")
        print(f"    слово: {to_word(decoded)}" + ("" if to_word(decoded) == word else "  <- слово испорчено"))
```

Листинг 4 – Задание 3. Код Хэмминга